JSQKV: Joint Sparsification and Quantization for KV-Cache Compression and Decode Acceleration

First Author[1][0000−1111−2222−3333], Second Author[2,3][1111−2222−3333−4444], and Third Author[3][2222−3333−4444−5555]

[1] Princeton University, Princeton NJ 08544, USA
[2] Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com
http://www.springer.com/gp/computer-science/lncs
[3] ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Abstract. As long-context large language models become increasingly widespread, autoregressive decoding is increasingly bottlenecked by KV-cache footprint and memory bandwidth. Existing KV-cache compression methods typically rely on sparsification or low-bit quantization, but simply combining the two often fails to simultaneously preserve model accuracy and deliver real system-level gains. We present JSQKV, a joint sparse-quant framework for KV-cache compression and decode acceleration that co-designs compression policy, numerical representation, and execution path. JSQKV introduces an attention-score-based differential sparsification strategy with a dual-window online execution mechanism, together with a Hadamard-based Per-Token-Tile quantization scheme and a bitmap-based sparse-quant format with a matching decode kernel. This co-design enables compressed KV-cache data to remain compact in memory while being effectively translated into reduced memory traffic and end-to-end speedup. Experimental results show that, under matched compression budgets, JSQKV generally compares favorably with the sequential MUSTAFAR+KIVI baseline on LongBench, with larger gains under more aggressive sparse-quant settings. On Meta-Llama-3-8B-Instruct, with input length 4096 and output length 256, JSQKV improves end-to-end throughput by 44.2% over the dense baseline at batch size 4 using 70% KV sparsity and 2-bit quantization, while increasing the KV-cache compression ratio to 3.48×. These results suggest that efficient KV-cache compression benefits from joint optimization across compression policy, numerical representation, and execution path. Code is available at https://github.com/Haozon/JSQKV.

Keywords: KV-cache compression · Sparsity · Quantization

1 Introduction

As context windows continue to grow in large language models [15, 6], autoregressive decoding is increasingly bottlenecked by KV-cache footprint and mem-

ory bandwidth [11, 21]. The KV cache grows linearly with context length, while decoding offers limited parallelism and repeatedly accesses the KV cache, making it more memory-bound than the prefill stage [11, 1, 21]. In long-context settings, this bottleneck further limits throughput, batch size, and deployment efficiency [11, 1].

Prior work on KV-cache optimization mainly follows two directions. One line reduces the amount of retained history through token eviction or sparsification. Representative methods such as StreamingLLM [17], H_2O [20], SnapKV [12], and PyramidKV [4] retain only selected historical tokens, while ThinK [18], KVPruner [14], and MUSTAFAR [10] further explore channel-wise, structured, and unstructured sparsity. These studies show that historical tokens are not equally important and that importance-aware cache reduction can effectively relieve memory and bandwidth pressure. The other line compresses the numerical representation of KV-cache elements through low-bit quantization. KVQuant [8], QuaRot [2], and KIVI [13] explore ultra-low-bit KV representation, distribution reshaping, and asymmetric quantization for Keys and Values. These methods directly reduce KV-cache footprint and bandwidth cost, but they mainly focus on how to quantize each cache element rather than which historical information should be retained.

Overall, prior work shows that both sparsification and quantization are effective, but the two are still largely studied in isolation. In practice, joint compression is not a simple sequential combination: once sparsification changes the retained tokens and cache access pattern, the appropriate quantization granularity, compressed data layout, and decode-time kernel design also change. Without such co-design, algorithm-level compression gains may not reliably translate into practical end-to-end speedups [1, 21].

To address this issue, we propose JSQKV, a joint sparse-quant framework for KV-cache compression and decode acceleration. JSQKV jointly designs compression policy, numerical representation, and execution path. The key idea is simple: allocate different compression budgets to different tokens according to their importance to future attention, compress the remaining KV states with a token-aligned and numerically stable low-bit representation, and keep the KV cache compressed in memory as much as possible until dense tiles are required by the decode kernel. Through this joint design, JSQKV keeps the KV cache compact in memory while more effectively reducing memory traffic and improving end-to-end throughput.

The main contributions of this paper are as follows:

- We propose an attention-score-based differential sparsification policy that assigns different sparsity levels to different historical tokens at the token level, instead of applying a single global compression ratio.
- We design a dual-window online execution mechanism for autoregressive decoding, extending differential sparsification from the prefill stage to online decoding.

- We propose a Hadamard-based Per-Token-Tile quantization method to mitigate low-bit quantization instability caused by Key outliers and to better align quantization granularity with token-level compression decisions.
- We design a bitmap-based sparse-quant format and a matching decode kernel that keep the compressed KV cache compact in memory and turn compression into reduced memory traffic and end-to-end decoding speedup.

Experimental results show that JSQKV achieves a favorable balance between model quality and system efficiency under high compression ratios. On Meta-Llama-3-8B-Instruct [7], with 70% KV cache sparsity and 2-bit quantization, JSQKV improves end-to-end throughput by approximately 44.2% over the dense baseline in a long-context decoding setting with batch size 4. Under matched compression budgets, JSQKV generally compares favorably with the sequential MUSTAFAR+KIVI baseline, with larger gains under more aggressive sparse-quant settings. These results support the motivation of treating KV-cache compression as a joint optimization problem across compression policy, numerical representation, and execution path.

2 Method

2.1 Overview

Figure 1 summarizes the overall design of JSQKV. Rather than applying sparsification and quantization as two independent post-processing steps, JSQKV organizes them into a unified decode-stage pipeline in which the compression policy, low-bit representation, and execution path are designed together.

The method consists of four tightly coupled components. First, JSQKV uses prefill attention to estimate token importance and instantiates a budgeted differential sparsity policy, so that different historical tokens receive different retention budgets under a fixed average compression target. Second, because future attention is unavailable when a token is first generated during decoding, JSQKV introduces a dual-window online execution mechanism that delays compression until enough future evidence has been accumulated. Third, after token-level compression decisions are made, the retained KV states are stored in a token-aligned low-bit form using Hadamard-based Per-Token-Tile quantization, which improves robustness to key outliers while matching the downstream sparse-quant execution granularity. Finally, JSQKV uses a bitmap-based sparse-quant format and a matching decode kernel to keep the KV cache compact in memory and turn compression into reduced memory traffic and practical decode-time speedup.

The remaining subsections describe these components in order. Section 2.2 presents the budgeted differential sparsity policy, Section 2.3 introduces the dual-window online execution mechanism, Section 2.4 describes the token-aligned quantization design, and Section 2.5 presents the sparse-quant storage format and decode kernel.

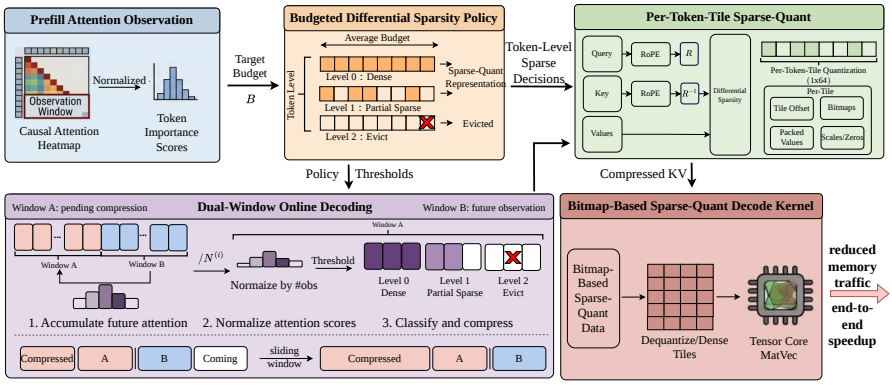


Fig. 1. Overview of JSQKV. The method first estimates token importance from prefill attention and instantiates a budgeted differential sparsity policy. It then extends this policy to autoregressive decoding through a dual-window online execution mechanism, represents retained KV states with Hadamard-based Per-Token-Tile quantization, and finally executes the compressed cache with a bitmap-based sparse-quant decode kernel to reduce memory traffic.

2.2 Budgeted Differential Sparsity Policy

The first challenge in KV-cache compression is to allocate a fixed compression budget across historical tokens with different future importance. A uniform sparsity ratio treats all tokens equally, even though some tokens carry persistent evidence for future decoding while others are rarely used again. Under the same average budget, such a uniform policy can waste budget on unimportant tokens and over-compress important ones. To address this issue, JSQKV estimates token importance from prefill attention and instantiates a three-level sparsity policy under an explicit average-budget constraint.

We use the attention distribution in prefill to estimate the relative importance of historical tokens. Let $\mathbf{A} \in \mathbb{R}^{T_q \times T_k}$ denote the causal attention matrix of the prompt. Following the observation-window idea in SnapKV [12], we use the last L_{obs} prompt tokens as an observation window. For a prefix token $j \le T_k - L_{\text{obs}}$, its importance score is defined as

$$s_j = \frac{T_k}{L_{\text{obs}}} \sum_{i=T_k-L_{\text{obs}}+1}^{T_k} A_{i,j}. \tag{1}$$

This score measures how much token j is attended by the queries in the observation window. Tokens inside the observation window are treated as highly important and are preserved by construction. The normalization factor T_k/L_{obs} makes the scores more comparable across prompts of different lengths.

Given the importance scores $\{s_j\}$, JSQKV assigns each token to one of three compression levels. Let $[p_0, p_1, p_2]$ denote the target proportions of Level 0, Level 1, and Level 2 tokens, where

$$p_0 + p_1 + p_2 = 1. \tag{2}$$

We compute two percentile thresholds,

$$\tau_{\text{high}} = \text{Percentile}(s, 1 - p_0), \tag{3}$$
$$\tau_{\text{low}} = \text{Percentile}(s, p_2), \tag{4}$$

and assign

$$\text{level}(j) = \begin{cases} 0, & s_j \geq \tau_{\text{high}}, \\ 1, & \tau_{\text{low}} \leq s_j < \tau_{\text{high}}, \\ 2, & s_j < \tau_{\text{low}}. \end{cases} \tag{5}$$

The three levels correspond to different compression strengths. Level 0 tokens are kept dense. Level 2 tokens are fully removed from the cache. Level 1 tokens use feature-level sparsity and keep only the largest-magnitude features in their key and value vectors. For a key vector $\mathbf{k}_j \in \mathbb{R}^d$, we apply

$$\tilde{k}_{j,i} = \begin{cases} k_{j,i}, & i \in \text{TopK}(|\mathbf{k}_j|, \lfloor (1 - \rho_1)d \rfloor), \\ 0, & \text{otherwise}, \end{cases} \tag{6}$$

where ρ_1 is the feature-level sparsity ratio for Level 1 tokens. The same rule is applied to $\mathbf{v}_j$. In this way, important tokens retain more information, while less important tokens absorb more of the compression budget.

A fixed average sparsity budget does not uniquely determine the three-level policy. Different dense/partial/evict allocations can share the same average compression ratio but lead to different accuracy outcomes. We therefore instantiate the policy under an explicit budget constraint. Let

$$c = ([p_0, p_1, p_2], \rho_1) \tag{7}$$

denote one candidate configuration. With $\rho_0 = 0$ for Level 0 and $\rho_2 = 1$ for Level 2, the target average sparsity budget B satisfies

$$B = p_0\rho_0 + p_1\rho_1 + p_2\rho_2 = p_1\rho_1 + p_2. \tag{8}$$

For a given target budget B, we search over a small feasible set of (p_0, ρ_1) pairs and analytically recover the remaining proportions:

$$p_1 = \frac{1 - p_0 - B}{1 - \rho_1}, \qquad p_2 = 1 - p_0 - p_1. \tag{9}$$

Candidates with invalid proportions are discarded. We then select the final configuration on a small calibration split:

$$c^\star = \arg\max_{c \in \mathcal{C}(B)} \text{Metric}(D_{\text{cal}}; c), \tag{10}$$

where $\mathcal{C}(B)$ is the feasible candidate set under budget B. This procedure keeps the overall compression target fixed while allowing JSQKV to allocate the budget more carefully across dense, partially sparse, and evicted tokens.

2.3 Dual-Window Online Execution Mechanism

The policy in Section 2.2 relies on future attention observations that are available in prefill but unavailable when a token is first generated during decoding. As a result, the token-level compression policy cannot be applied immediately after a token is written into the KV cache. To make the same policy work during autoregressive decoding, JSQKV introduces a dual-window online execution mechanism that delays compression until each token has accumulated enough future attention evidence.

During decoding, the KV cache is organized into two consecutive windows of size W, as illustrated in Fig. 2. Window A stores tokens waiting to be compressed, while Window B stores newly generated tokens whose queries provide additional attention observations for Window A. This design avoids compressing a token too early, before it has accumulated enough future attention observations.

When a new query $\mathbf{q}_t$ is produced at decoding step t, we compute its attention to the tokens in Window A:

$$\boldsymbol{\alpha}_t = \mathrm{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_A^\top}{\sqrt{d}}\right), \tag{11}$$

where $\mathbf{K}_A$ denotes the key states of Window A. The resulting attention scores are accumulated in an attention buffer

$$\mathbf{Acc} \in \mathbb{R}^{N_b \times H \times W}, \tag{12}$$

where N_b is the batch size, H is the number of attention heads, and W is the window size.

When Window B becomes full, tokens in Window A have received different numbers of future observations, since earlier tokens are visible to more later queries. We therefore normalize the accumulated attention before reusing it as the token-importance signal. For the token at position $i \in \{0, \ldots, W-1\}$ in Window A, where i is zero-indexed, the number of observations is

$$N^{(i)} = 2W - 1 - i, \tag{13}$$

which yields the normalized score

$$\bar{\alpha}^{(i)} = \frac{\mathbf{Acc}[:, :, i]}{N^{(i)}}. \tag{14}$$

We then compare the normalized scores against the thresholds $(\tau_{\text{high}}, \tau_{\text{low}})$ from Section 2.2. Each token in Window A is assigned to Level 0, Level 1, or Level 2 accordingly.

After the classification, JSQKV applies the corresponding compression rule to Window A: Level 0 tokens remain dense, Level 1 tokens are partially pruned, and Level 2 tokens are removed. The compressed Window A is then merged into the historical compressed region, the previous Window B becomes the new Window A, the accumulator is reset, and decoding continues with an empty new Window B. In this way, each token is observed before compression while the number of uncompressed tokens remains bounded by $O(W)$.

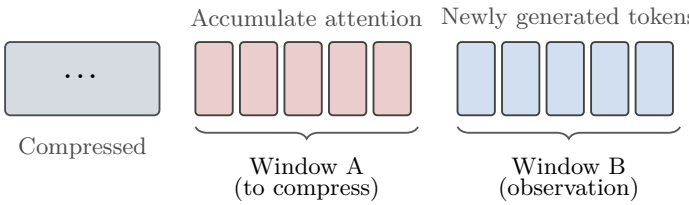


Fig. 2. Dual-window online execution mechanism during decoding. Window A accumulates attention statistics before compression, while Window B collects newly generated tokens and provides future observations.

2.4 Per-Token-Tile Quantization with Hadamard Rotation

The remaining nonzero KV states still require a low-bit representation to reduce storage and memory traffic. Since token-level compression decisions are carried through the downstream sparse-quant operators, which are likewise organized as tiles along the token dimension, quantization should follow the same granularity. We therefore adopt *Per-Token-Tile* quantization, which aligns the quantization unit with both token-level compression decisions and the tile-based decode path, while providing a fine-grained low-bit representation.

For a token vector $\mathbf{x}_t \in \mathbb{R}^d$ (either key or value), the feature dimension is partitioned into $M = d/g$ contiguous tiles of size g:

$$\mathbf{x}_t = [\mathbf{x}_{t,1}, \mathbf{x}_{t,2}, \ldots, \mathbf{x}_{t,M}], \qquad \mathbf{x}_{t,m} \in \mathbb{R}^g. \tag{15}$$

Each tile is quantized independently with its own scale $s_{t,m}$ and zero point $z_{t,m}$. The quantized tile is computed as

$$\mathbf{q}_{t,m} = \mathrm{clip}\left(\mathrm{round}\left(\frac{\mathbf{x}_{t,m} - z_{t,m}}{s_{t,m}}\right), 0,\ 2^b - 1\right), \tag{16}$$

where b is the quantization bitwidth. This design keeps the quantization parameters aligned with the token-level compression decision while reducing the local dynamic range within each tile.

A remaining challenge is that key states often contain outlier values, which can dominate the quantization range and make aggressive low-bit quantization unstable. To mitigate this effect, JSQKV applies a Hadamard rotation to queries and keys before quantization. Let $\mathbf{H} \in \mathbb{R}^{d\times d}$ denote the normalized Hadamard matrix. We rotate

$$\tilde{\mathbf{q}}_i = \frac{1}{\sqrt{d}}\mathbf{H}\mathbf{q}_i, \qquad \tilde{\mathbf{k}}_j = \frac{1}{\sqrt{d}}\mathbf{H}\mathbf{k}_j. \tag{17}$$

Because the Hadamard matrix is orthogonal, the attention score is preserved:

$$\tilde{\mathbf{q}}_i^\top \tilde{\mathbf{k}}_j = \mathbf{q}_i^\top \mathbf{k}_j. \tag{18}$$

The rotation therefore improves the distribution seen by the quantizer without changing the attention computation itself.

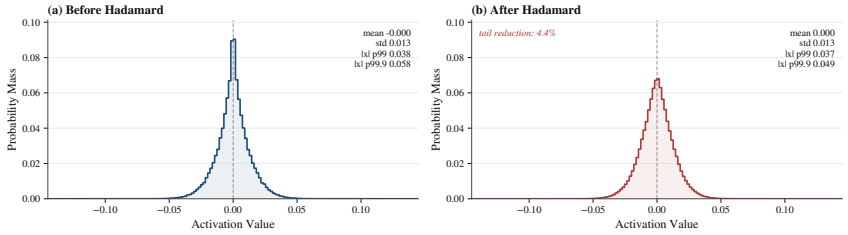


Fig. 3. Key-state activation distribution before and after Hadamard rotation on a LLaMA2-7B sample. The rotation suppresses heavy tails while preserving enough variation for feature-level sparsification.

Table 1. PPL of RTN and Hadamard rotation (*Rotate*) on WikiText-2 under quantization-only and sparse-quant settings. Hadamard rotation improves low-bit quantization stability, while remaining compatible with subsequent KV sparsification.

	4-bit		3-bit		2-bit	
Setting	RTN	Rotate	RTN	Rotate	RTN	Rotate
Quant only	5.22	**5.14**	6.57	**5.28**	115.55	**5.35**
+50% KV sparsity	**5.27**	5.69	6.57	**6.35**	115.55	**6.65**

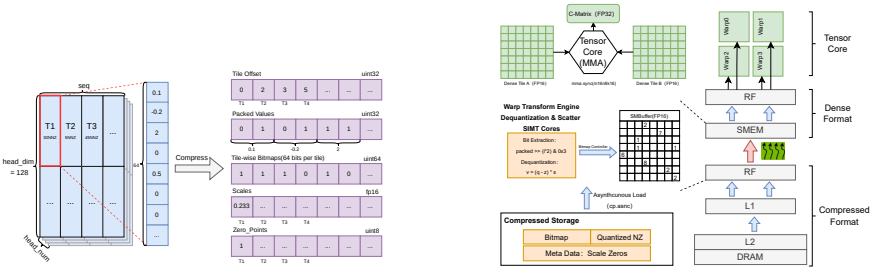
Figure 3 illustrates the effect of the Hadamard rotation on key states. After rotation, the heavy tail is noticeably suppressed, which reduces the dominance of outliers in low-bit quantization. At the same time, the rotated distribution does not collapse into a fully uniform one, and still preserves sufficient magnitude variation for subsequent magnitude-based feature sparsification.

Table 1 further supports this observation quantitatively on WikiText-2. Under quantization-only settings, Hadamard rotation improves PPL at 4-bit, 3-bit, and especially 2-bit, supporting its role in stabilizing low-bit quantization. When 50% KV sparsity is further applied, the same trend remains visible at 3-bit and 2-bit, where Rotate still outperforms RTN by a meaningful margin. At 4-bit with added sparsity, however, the gain becomes marginal and slightly reverses, suggesting that when quantization error is already relatively small, the rotation-induced mixing may mildly weaken the magnitude separability used by subsequent sparsification. Overall, Per-Token-Tile quantization provides a fine-grained low-bit representation, while Hadamard rotation appears to improve robustness to key outliers without largely disrupting subsequent sparsification.

2.5 Sparse-Quant Format and Decode Kernel

Compression quality alone is not enough if the resulting representation cannot be executed efficiently at decode time. We therefore co-design a sparse-quant format and its matching decode kernel, so that the compressed KV cache not only reduces storage cost, but also translates into practical decode acceleration.

Figure 4 gives an overview of this design. The left panel shows the sparse-quant format. Each 1×64 tile stores a bitmap for nonzero positions, packed



(a) bitmap-based sparse-quant format (b) Sparse-quant decode kernel

Fig. 4. Overview of the sparse-quant format and decode kernel. The left panel shows the bitmap-based sparse-quant format. The right panel shows the decode kernel that loads compressed tiles, performs unpacking and dequantization, reconstructs dense shared-memory tiles, and invokes Tensor Core matrix-vector computation.

low-bit values, a per-tile offset, and quantization metadata such as scales and zero points. This layout is designed to support high sparsity and low-bit storage while preserving a regular memory organization, thereby avoiding the irregular accesses and indexing overheads of conventional sparse formats.

The right panel shows the matching decode kernel. During decode, the kernel follows a *load-as-compressed, compute-as-dense* strategy. It first loads bitmaps and packed low-bit values from memory, then reconstructs the nonzeros through unpacking and dequantization, and scatters them into dense shared-memory tiles. Tensor Core matrix-vector computation is then performed on these reconstructed dense tiles. In this way, sparsity is handled only along the data-loading path, while the actual compute stage still uses regular dense operators.

This design is particularly important for decode-stage attention, which is strongly memory-bound. By keeping the KV cache compressed in memory, the format reduces memory traffic; by reconstructing dense tiles only along the compute path, the kernel avoids the control-flow divergence and irregular accesses that often limit sparse execution. As a result, the sparse-quant format and decode kernel together convert compressed KV representations into practical decode-time gains, rather than merely reducing storage footprint.

3 Experiments

3.1 Experimental Setup

We evaluate JSQKV on both accuracy and efficiency. Our implementation is built on Python 3.10, PyTorch 2.6.0, CUDA 12.4, and FlashAttention [5], and uses custom CUDA/Triton kernels for sparse-quant packing and the sparse-quant decode kernel. All experiments are conducted on a single NVIDIA A100 80GB PCIe GPU with an Intel Xeon CPU and 252GB host memory.

For accuracy evaluation, we use LongBench [3] with the official pipeline. Our main sparse-only and joint sparse-quant comparisons focus on six focal LongBench tasks—NarrativeQA, Qasper, MultiFieldQA-EN, HotpotQA, TREC, and

LCC—which cover diverse long-context behaviors while enabling stable matched-budget comparisons across compression settings and model families. We use Meta-Llama-3-8B-Instruct [7] as the main model and further evaluate on Llama-2-7B [16], Mistral-7B-v0.1 [9], and Qwen2.5-7B-Instruct [19]. Inputs are truncated to 8192 tokens by keeping the first 4096 and the last 4096 tokens when necessary. We compare against dense decoding, a MUSTAFAR-style uniform sparsity baseline [10], and a sequential sparse-then-quantize baseline that combines uniform sparsity with KIVI-style low-bit quantization [13], denoted as **M+K**. We report 50% and 70% average sparsity for sparse-only comparisons, and 50/50 and 70/70 K/V sparsity with 4-bit and 2-bit quantization for joint sparse-quant comparisons. Differential policies are selected by a lightweight held-out calibration procedure; detailed reporting rules and finalized configurations are provided in Appendix C.

For efficiency evaluation, we use Meta-Llama-3-8B-Instruct [7] with 4096 input tokens, 256 generated tokens, and batch sizes from 1 to 8. We report end-to-end decoding throughput, compression overhead, compressed KV-cache size, and compression ratio. Runtime comparisons focus on executable baselines, namely dense decoding, the MUSTAFAR-style sparse kernel, and the proposed sparse-quant runtime path.

3.2 Accuracy Results

Differential Sparsity Alone We first study differential sparsity without quantization or Hadamard rotation, in order to isolate the effect of budget allocation. We compare JSQKV against a matched-budget *uniform* sparsity baseline, where all tokens are compressed with the same sparsity rule regardless of importance. This is also the sparse token policy used in MUSTAFAR-style KV-cache compression. We report results at 50% and 70% average sparsity, with dense decoding as an uncompressed reference.

For each task and target budget, we select the differential policy on a small calibration split and re-check it on a disjoint validation split before full evaluation. For robustness, we use a conservative reporting protocol, and the finalized reported results are summarized in the main text; detailed task-level outcomes and reporting decisions are provided in Appendix C.

Table 3.2 reports sparse-only results on six focal LongBench tasks for Meta-Llama-3-8B-Instruct [7]. The main comparison is between matched-budget uniform and differential sparsity. Under the conservative reporting protocol, the finalized differential results remain non-negative, and the gain is more pronounced at 70% sparsity than at 50%. This shows that differential budget allocation becomes increasingly valuable under tighter sparsity budgets.

The same pattern holds across architectures. Table 3 summarizes sparse-only averages on three representative models. In all three cases, differential sparsity remains non-negative in the finalized results, and the improvement is consistently larger at 70% sparsity than at 50%. Detailed task-level results for the additional models, together with the corresponding finalized differential-sparsity configurations, are provided in Appendix C.

Table 2. Sparse-only differential sparsity versus matched-budget uniform sparsity on Meta-Llama-3-8B-Instruct [7] over six focal LongBench tasks. Dense decoding is reported as an uncompressed reference. "Uniform" corresponds to the token-agnostic policy used in MUSTAFAR. In a few cases, the reported differential result reverts to the uniform baseline.

Method	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
Dense	23.45	42.18	43.14	46.06	74.00	57.12	47.66
Uniform-50	23.44	43.73	42.93	45.94	73.50	56.03	47.59
Differential-50	23.44	43.73	42.93	45.94	**74.00**	**57.08**	**47.85**
Uniform-70	23.94	40.89	40.91	44.93	70.00	54.12	45.80
Differential-70	23.94	**41.03**	**41.61**	**45.12**	**72.50**	**55.45**	**46.61**

Table 3. Cross-model average sparse-only results on the six focal LongBench tasks. "Differential" denotes the finalized full-task average under the same reporting protocol as the main-text sparse-only study.

Model	Budget	Uniform Avg.	Differential Avg.	Δ
Meta-Llama-3-8B-Instruct	50%	47.59	**47.85**	+0.26
Meta-Llama-3-8B-Instruct	70%	45.80	**46.61**	+0.81
Llama-2-7B	50%	30.88	**31.54**	+0.66
Llama-2-7B	70%	29.29	**30.78**	+1.49
Qwen2.5-7B-Instruct	50%	31.56	**31.68**	+0.11
Qwen2.5-7B-Instruct	70%	30.67	**31.24**	+0.57

Overall, these sparse-only results show that differential sparsity is already beneficial before introducing quantization, and that its advantage grows with sparsity. This motivates the full JSQKV design: if token selection already matters in the sparse-only regime, it should matter even more once low-bit representation and execution-path constraints are introduced.

Joint sparsity-quantization results. We evaluate the full joint sparse-quant framework on six focal LongBench tasks: NarrativeQA, Qasper, MultiFieldQA-EN, HotpotQA, TREC, and LCC. Table 4 reports detailed results on Meta-Llama-3-8B-Instruct [7]. On this model, JSQKV yields higher average scores than the sequential M+K baseline in all four tested settings. The gain is modest under milder compression, but becomes larger under more aggressive sparse-quant settings, especially at 70/70 + 2-bit. Under this most aggressive setting, the average score improves from 39.83 to 43.12, with notable gains on Qasper, TREC, and LCC. This trend is consistent with the motivation of JSQKV: as the compression ratio increases, the benefit of jointly aligning sparsity, quantization, and runtime execution also increases.

To test whether the trend is specific to one architecture, Table 5 summarizes the average score on the same six focal LongBench tasks for three additional model families. The results indicate a generally favorable but not perfectly uniform trend for JSQKV over M+K across models and compression settings. The gains are most visible on Meta-Llama-3-8B-Instruct and Mistral-7B-v0.1, and remain visible in most settings on Llama-2-7B and Qwen2.5-7B-Instruct, espe-

Table 4. Detailed results on six focal LongBench tasks for Meta-Llama-3-8B-Instruct [7]. "50/50" and "70/70" denote target sparsity ratios for K and V caches.

Setting	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
M+K, 50/50 + 4-bit	21.61	**40.70**	**47.62**	41.13	**70.50**	53.83	45.90
JSQKV, 50/50 + 4-bit	**21.92**	39.38	46.77	**41.38**	**70.50**	**55.64**	**45.93**
M+K, 50/50 + 2-bit	**21.97**	38.96	**46.11**	40.39	68.00	44.86	43.38
JSQKV, 50/50 + 2-bit	21.16	**39.54**	44.49	**41.19**	**71.50**	**46.26**	**44.02**
M+K, 70/70 + 4-bit	20.94	37.32	44.90	**41.22**	68.00	52.71	44.18
JSQKV, 70/70 + 4-bit	**21.86**	**38.81**	**46.52**	40.64	**70.00**	**54.23**	**45.34**
M+K, 70/70 + 2-bit	**21.34**	35.43	43.43	39.21	63.00	36.59	39.83
JSQKV, 70/70 + 2-bit	19.70	**39.49**	**44.31**	**40.15**	**70.00**	**45.04**	**43.12**

Table 5. Cross-model averages on six focal LongBench tasks. "M+K" denotes the sequential MUSTAFAR+KIVI baseline.

	50/50 + 4-bit		50/50 + 2-bit		70/70 + 4-bit		70/70 + 2-bit	
Model	M+K	JSQKV	M+K	JSQKV	M+K	JSQKV	M+K	JSQKV
Meta-Llama-3-8B-Instruct	45.90	**45.93**	43.38	**44.02**	44.18	**45.34**	39.83	**43.12**
Llama-2-7B	31.34	**31.37**	**31.51**	30.23	30.57	**30.97**	29.92	**30.09**
Mistral-7B-v0.1	32.84	**32.90**	31.70	**32.37**	32.43	**32.71**	30.14	**32.35**
Qwen2.5-7B-Instruct	31.17	**37.08**	**26.10**	23.48	33.00	**39.67**	25.51	**35.20**

cially under tighter budgets. At the same time, a few settings remain negative or nearly neutral, such as Llama-2-7B at 50/50 + 2-bit and Qwen2.5-7B-Instruct at 50/50 + 2-bit. Overall, these results suggest that the benefit of JSQKV is mainly associated with the joint design itself, while its magnitude still depends on model family and compression budget. To isolate the quantization contribution from the sparse policy, Appendix B further reports a quantization-only comparison between the JSQKV quantizer and a matched KIVI-style baseline.

3.3 Efficiency Results

Before presenting runtime results, we clarify the efficiency comparison protocol. In the accuracy experiments, we include **M+K** (MUSTAFAR + KIVI) as a sequential sparse-plus-quantization baseline to evaluate compression quality under matched budgets. However, this sequential composition is an algorithmic baseline rather than a jointly executable runtime path: while MUSTAFAR provides a sparse decode kernel, a naive combination with KIVI does not by itself define a unified sparse-quant execution kernel for end-to-end decoding. In contrast, a central contribution of JSQKV is the algorithm–system co-design of compression policy, low-bit representation, storage format, and decode kernel, which allows sparsity and quantization to be executed together and translated into practical system-level gains. Therefore, in the efficiency experiments, we compare JSQKV primarily against dense decoding and the MUSTAFAR-style sparse kernel, which provides the closest executable baseline.

End-to-end throughput. Table 6 and Fig. 5 report end-to-end decoding throughput on Meta-Llama-3-8B-Instruct [7]. Dense decoding is strongest at

Table 6. End-to-end decoding throughput (tokens/s) on Meta-Llama-3-8B-Instruct [7] with 4096 input tokens and 256 output tokens.

Batch Size	Dense	MUSTAFAR-50	MUSTAFAR-70	JSQKV 50/50 + 2-bit	JSQKV 70/70 + 2-bit
1	28.55	17.16	16.84	23.24	23.38
2	39.81	31.75	30.56	42.76	42.37
3	49.15	45.15	46.18	55.45	57.81
4	55.83	53.80	57.63	64.70	66.42
5	56.67	66.93	65.96	72.02	74.22
6	56.81	77.17	78.01	76.80	80.38
7	59.69	86.52	83.53	80.47	84.06
8	62.80	94.51	96.43	86.98	87.12

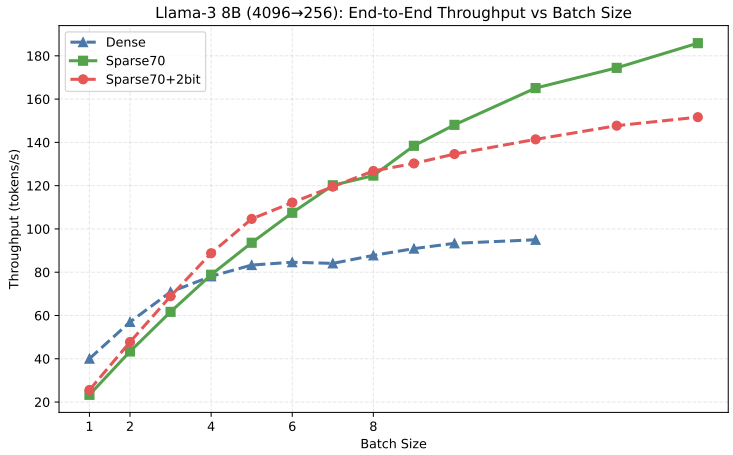


Fig. 5. End-to-end decoding throughput on Meta-Llama-3-8B-Instruct [7] with 4096 input tokens and 256 output tokens. The full sparse-quant path provides its most visible advantage in the small-to-medium batch regime.

batch size 1, where the compression overhead is not yet amortized. As batch size increases, the compressed paths become increasingly favorable, indicating that the workload becomes more bandwidth-sensitive. The most visible advantage of the full sparse-quant design appears in the small-to-medium batch regime. For example, under the 70/70 + 2-bit setting, JSQKV reaches 80.38 tokens/s at batch size 6, which is 41.5% higher than dense decoding and 3.0% higher than MUSTAFAR-70. At larger batch sizes, however, the throughput gap narrows and sparsity-only configurations can become preferable, suggesting that the extra unpacking and dequantization overhead is less effectively amortized once the workload becomes more compute-heavy. Overall, these results suggest that joint sparse-quant compression can translate into practical decoding speedups when KV-cache traffic remains the dominant bottleneck.

Compression overhead. Compression is beneficial only when its own runtime cost remains controlled. Table 7 reports representative compression statistics on Meta-Llama-3-8B-Instruct [7]. The results show that the sparse-quant

Table 7. Representative compression overhead and KV-cache size for Meta-Llama-3-8B-Instruct [7] (input length 4096). Compression ratio is original KV size divided by compressed KV size.

Batch Size	Method	KV-Cache Size (MB)	Compression Ratio	Compression Time (ms)
1	Sparse50	2720.16	1.51×	740.80
	Sparse70	1964.29	2.09×	716.30
	Sparse50+2bit	1261.16	3.25×	545.18
	Sparse70+2bit	1176.25	3.48×	552.86
8	Sparse50	2720.16	1.51×	5688.23
	Sparse70	1964.29	2.09×	5411.29
	Sparse50+2bit	1261.16	3.25×	4317.88
	Sparse70+2bit	1176.25	3.48×	4293.18

path achieves substantially higher compression ratios than sparsity-only baselines while keeping compression time competitive. For example, at batch size 8, MUSTAFAR-70 reduces the KV cache to 1964.29MB, whereas JSQKV under the 70/70 + 2-bit setting further reduces it to 1176.25MB, improving the compression ratio from 2.09× to 3.48×, while also lowering compression time from 5411.29ms to 4293.18ms. This suggests that the benefit of JSQKV is not limited to reduced decode-time bandwidth demand; the more compact sparse-quant representation also reduces compression-side write-back cost.

4 Discussion and Future Work

Although JSQKV achieves a favorable trade-off between accuracy and efficiency in long-context settings, several limitations remain for future work.

First, the current method relies on explicit token-importance estimation rather than a uniform sparsity budget, but this modeling still mainly operates at the token level. It does not yet capture finer-grained sensitivity across layers, attention heads, or Key/Value representations. A more fine-grained joint design of importance modeling and quantization is therefore worth exploring.

Second, the joint sparse-quant path is not advantageous under all serving conditions. Its largest throughput gains appear in small- to medium-batch scenarios, whereas at larger batch sizes the benefit of compression is increasingly offset by dequantization, metadata handling, and higher compute pressure. Designing more efficient sparse-quant operators thus remains an important direction.

Finally, the current evaluation scope is still limited. Although the experiments cover multiple model families and representative long-context settings, further study is needed to verify whether the proposed method remains effective in ultra-long-context scenarios, on heterogeneous accelerators, and in production-like deployment environments.

5 Conclusion

JSQKV reduces KV-cache cost through differential sparsification, stabilized low-bit quantization, and a bitmap-based sparse-quant decode kernel. Evaluation

results show that, under matched compression budgets, JSQKV generally compares favorably with a sequential sparsity-plus-quantization baseline, with larger gains under more aggressive sparse-quant settings. They further show that compression savings can translate into practical throughput gains during decoding. Overall, these findings support treating KV-cache compression as a systems-and-algorithms co-design problem rather than as a simple composition of independent post-processing steps.

References

1. Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B.S., Tumanov, A., Ramjee, R.: Taming throughput-latency tradeoff in llm inference with sarathi-serve. arXiv preprint arXiv:2403.02310 (2024), https://arxiv.org/abs/2403.02310
2. Ashkboos, S., Mohtashami, A., Croci, M.L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., Hensman, J.: Quarot: Outlier-free 4-bit inference in rotated llms. arXiv preprint arXiv:2404.00456 (2024)
3. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508 (2023), https://arxiv.org/abs/2308.14508
4. Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., Xiao, W.: Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. arXiv preprint arXiv:2406.02069 (2024), https://arxiv.org/abs/2406.02069
5. Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Re, C.: Flashattention: Fast and memory-efficient exact attention with io-awareness. Advances in Neural Information Processing Systems **35** (2022)
6. Ding, Y., Zhang, L.L., Zhang, C., Xu, Y., Shang, N., Xu, J., Yang, F., Yang, M.: Longrope: Extending llm context window beyond 2 million tokens. arXiv preprint arXiv:2402.13753 (2024), https://arxiv.org/abs/2402.13753
7. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
8. Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M.W., Shao, Y.S., Keutzer, K., Gholami, A.: Kvquant: Towards 10 million context length llm inference with kv cache quantization. arXiv preprint arXiv:2401.18079 (2024)
9. Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., Casas, D.d.l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al.: Mistral 7b. arXiv preprint arXiv:2310.06825 (2023), https://arxiv.org/abs/2310.06825
10. Joo, D., Hosseini, H., Hadidi, R., Asgari, B.: Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference. arXiv preprint arXiv:2505.22913 (2025), https://arxiv.org/abs/2505.22913
11. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (2023), https://arxiv.org/abs/2309.06180
12. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., Chen, D.: Snapkv: Llm knows what you are looking for before generation.

arXiv preprint arXiv:2404.14469 (2024), https://arxiv.org/abs/2404.14469, version 2, June 17, 2024
13. Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., Hu, X.: Kivi: A tuning-free asymmetric 2bit quantization for kv cache. arXiv preprint arXiv:2402.02750 (2024)
14. Lv, B., Zhou, Q., Ding, X., Wang, Y., Ma, Z.: Kvpruner: Structural pruning for faster and memory-efficient large language models. arXiv preprint arXiv:2409.11057 (2024), https://arxiv.org/abs/2409.11057
15. Peng, B., Quesnelle, J., Fan, H., Shippole, E.: Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071 (2023), https://arxiv.org/abs/2309.00071
16. Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023), https://arxiv.org/abs/2307.09288
17. Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M.: Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453 (2024), https://arxiv.org/abs/2309.17453
18. Xu, Y., Jie, Z., Dong, H., Wang, L., Lu, X., Zhou, A., Saha, A., Xiong, C., Sahoo, D.: Think: Thinner key cache by query-driven pruning. In: International Conference on Learning Representations (2025), https://arxiv.org/abs/2407.21018, spotlight
19. Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Li, C., Zhang, D., Liu, F., Huang, H., Wei, H., et al.: Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 (2024), https://arxiv.org/abs/2412.15115
20. Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., Chen, B.: H_2o: Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048 (2023), https://arxiv.org/abs/2306.14048
21. Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H.: Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24) (2024), https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

A Threshold Stability for Prefill-to-Decode Reuse

Section 2.2 estimates the token-importance thresholds $(\tau_{\text{high}}, \tau_{\text{low}})$ from normalized prefill attention, and Section 2.3 reuses the same thresholds during decoding after normalizing the accumulated attention by the number of available future observations. This reuse strategy relies on a simple layer-wise stability assumption: under the same normalization rule, the importance-score scale of a fixed layer should remain sufficiently consistent for prefill-estimated thresholds to remain meaningful during online decoding.

We test this assumption using the exact threshold type employed by JSQKV. We compute normalized attention-derived importance scores from the observed attention statistics and derive the corresponding thresholds $(\tau_{\text{high}}, \tau_{\text{low}})$. The evaluation uses Meta-Llama-3-8B-Instruct [7] on seven datasets: WikiText-2, GSM8K, Qasper, MultiFieldQA-EN, NarrativeQA, HotpotQA, and MuSiQue. Each dataset contributes 10 samples, with maximum length 192 and representative layers 10 and 20.

Table 8 reports the resulting cross-dataset statistics. For Layer 10, the median CV is 10.21% for τ_{high} and 14.37% for τ_{low}; for Layer 20, the corresponding values are 19.40% and 19.82%. Figure 6 shows the full distributions. Two observations matter most. First, for a fixed layer, both thresholds remain concentrated within a bounded range across datasets. Second, the threshold scale is clearly layer dependent, so the supported claim is reuse of *fixed per-layer thresholds*, not a single global threshold shared by all layers.

These results are sufficient for the reuse mechanism in Section 2.3. The decoder does not require decode-time scores to match prefill thresholds exactly for every sample; it only requires the normalized scores to stay on a comparable layer-wise scale. Taken together, Table 8 and Figure 6 support estimating $(\tau_{\text{high}}, \tau_{\text{low}})$ once in prefill and reusing them during decoding without step-wise re-calibration.

Table 8. Cross-dataset stability of the normalized attention thresholds used by the prefill-to-decode reuse path. CV is computed from the per-dataset median thresholds.

Layer	Threshold	Median range	Median CV (%)
10	τ_{high}	0.67120–0.92427	10.21
10	τ_{low}	0.04243–0.06552	14.37
20	τ_{high}	0.35568–0.61690	19.40
20	τ_{low}	0.01732–0.02890	19.82

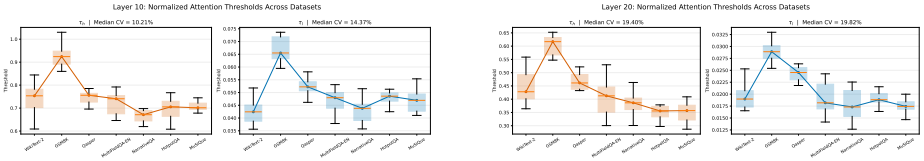


Fig. 6. Cross-dataset distributions of the normalized attention thresholds used by the prefill-to-decode reuse path. Left: Layer 10. Right: Layer 20. The thresholds remain stable at a fixed layer, while their absolute scale is layer dependent.

B Additional Quantization-Only Comparison

To isolate the contribution of the numerical representation in JSQKV, we further compare a quantization-only instantiation of the JSQKV quantizer against a matched KIVI-style baseline. This study is placed in the appendix because the main paper focuses on the full sparse-quant pipeline. In this comparison, we keep the same per-token-tile low-bit representation used by JSQKV but remove differential sparsity and the bitmap-based sparse runtime path.

Unless otherwise noted, both methods use a residual window of 128 tokens. The KIVI-style baseline uses channel-wise key quantization, per-token-head value quantization, and group size 128, whereas the JSQKV quantizer uses per-token-tile quantization with tile-wise Hadamard rotation (tile size 64 and Hadamard group size 64).

Table 9 reports detailed results on Meta-Llama-3-8B-Instruct [7] under the full 16-task LongBench protocol. At 4-bit and 3-bit, the two methods remain close in average score. At 2-bit, however, the JSQKV quantizer is noticeably stronger, improving the full-benchmark average from 26.94 to 30.59, with especially visible gains on HotpotQA and LCC. This pattern suggests that the tile-aligned design is most helpful in the aggressive low-bit regime.

Table 10 summarizes the same comparison on additional model families. To stay aligned with the main-text evaluation protocol, the cross-model comparison is anchored on the same six focal LongBench tasks used in the main sparse-only and joint sparse-quant results; full LongBench results are included only where those quantization-only runs are already available. Accordingly, this table is intended to show whether the quantization trend transfers across model families under the reported evaluation scopes, rather than to support strict cross-row ranking under a single unified benchmark. The JSQKV quantizer remains competitive on Llama-2-7B and again becomes favorable at 2-bit on Mistral-7B-v0.1.

Table 9. Quantization-only comparison on Meta-Llama-3-8B-Instruct [7]. "JSQKV quantizer" denotes per-token-tile quantization with tile Hadamard(64) and residual length 128. Avg. is the full 16-task LongBench average.

Setting	Bit	NarrativeQA	HotpotQA	GovReport	TREC	PCount	LCC	Avg.
KIVI-style	4-bit	**24.26**	46.06	29.53	**74.00**	**6.50**	54.07	**42.87**
JSQKV quantizer	4-bit	22.47	**46.07**	**30.03**	73.50	5.50	**57.20**	42.72
KIVI-style	3-bit	20.81	45.64	**29.35**	**74.00**	6.75	**60.94**	**41.80**
JSQKV quantizer	3-bit	**21.64**	**46.62**	29.16	73.50	**7.00**	53.66	41.48
KIVI-style	2-bit	16.55	27.50	23.91	**58.00**	2.00	27.03	26.94
JSQKV quantizer	2-bit	**17.90**	**37.93**	**25.63**	55.50	**3.00**	**30.37**	**30.59**

Table 10. Cross-model averages for the quantization-only comparison. Meta-Llama-3-8B-Instruct and Llama-2-7B use full LongBench; Mistral-7B-v0.1 uses the same six focal LongBench tasks as in the main text. Δ is "JSQKV quantizer minus KIVI-style".

Model	Protocol	KIVI 4-bit	JSQKV 4-bit	Δ	KIVI 3-bit	JSQKV 3-bit	Δ	KIVI 2-bit	JSQKV 2-bit	Δ
Meta-Llama-3-8B-Instruct	full LongBench	42.87	42.72	-0.15	41.80	41.48	-0.32	26.94	**30.59**	**+3.65**
Llama-2-7B	full LongBench	28.09	**28.33**	**+0.24**	**28.23**	27.82	-0.41	**24.33**	24.17	-0.16
Mistral-7B-v0.1	six focal LongBench tasks	**31.19**	30.78	-0.41	**30.16**	29.52	-0.64	23.91	**25.72**	**+1.81**

Overall, the cross-model results do not indicate a uniform advantage at 4-bit or 3-bit, but they suggest that the JSQKV quantizer becomes more favorable as the quantization budget is pushed to 2-bit. This trend is consistent with the main observation on Meta-Llama-3-8B-Instruct and supports the view that a main benefit of the proposed quantizer lies in improved robustness under aggressive low-bit quantization.

C Additional Sparse-Only Reporting Details

This appendix provides additional details for the sparse-only differential-sparsity study, including task-level results for the additional models and the corresponding finalized retained configurations. These materials are omitted from the main text for space reasons. As in the main text, all sparse-only results follow the same conservative reporting protocol: candidate differential policies are selected on a small calibration split, re-checked on a disjoint validation split, and reverted to the matched-budget uniform baseline when the differential policy does not yield a stable gain on the final full-task evaluation.

C.1 Task-Level Sparse-Only Results for Additional Models

Table 11, Table 12, Table 13, and Table 14 report finalized sparse-only task-level results for the additional models summarized in the main text. As in the main sparse-only study, "Differential" denotes the finalized reported result after applying the same conservative reporting protocol.

For Llama-2-13B [16] at 50% sparsity, the current run summary does not yet provide finalized full-task follow-ups for most focal tasks. We therefore do not include a task-level full-score table for that setting in the paper draft, and instead retain only the currently finalized average-level summary where available.

Table 11. Task-level sparse-only results on Llama-2-7B [16] over six focal LongBench tasks.

Method	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
Uniform-50	15.04	9.05	20.90	7.39	66.00	66.88	30.88
Differential-50	16.94	9.05	22.92	7.39	66.00	66.91	31.54
Uniform-70	13.66	7.69	19.42	6.57	64.50	63.89	29.29
Differential-70	15.00	7.88	22.73	6.96	65.50	66.59	30.78

Table 12. Task-level sparse-only results on Qwen2.5-7B-Instruct [19] over six focal LongBench tasks.

Method	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
Uniform-50	9.73	11.62	29.88	9.03	69.50	59.63	31.56
Differential-50	9.73	12.05	29.88	9.03	69.50	59.86	31.68
Uniform-70	8.60	11.04	28.52	8.76	68.00	59.11	30.67
Differential-70	9.33	11.72	28.61	9.68	69.00	59.11	31.24

Table 13. Task-level sparse-only results on Mistral-7B-v0.1 [9] over six focal LongBench tasks.

Method	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
Uniform-50	12.99	28.89	39.92	26.24	67.50	53.46	38.17
Differential-50	12.99	29.46	40.37	26.24	67.50	53.46	38.34
Uniform-70	13.38	26.64	38.83	26.67	66.50	53.44	37.58
Differential-70	13.38	26.98	40.65	26.67	67.00	53.44	38.02

Table 14. Task-level sparse-only results on Llama-2-13B [16] over six focal LongBench tasks at 70% sparsity.

Method	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
Uniform-70	7.03	5.62	13.52	13.91	51.00	57.59	24.78
Differential-70	11.19	6.97	17.46	13.91	70.00	65.11	30.77

Finalized Per-Task Reporting Decisions Table 15 summarizes, for each retained model and sparsity budget, how many of the six focal LongBench tasks ultimately keep a differential policy and how many revert to the matched-budget uniform baseline. This summary is intended to clarify the reporting protocol without overloading the main text with task-level bookkeeping.

Table 15. Summary of finalized sparse-only reporting decisions on the six focal LongBench tasks. "Differential retained" counts tasks whose final reported full-task result uses a differential policy; "Uniform revert" counts tasks whose final reported result reverts to the matched-budget uniform baseline.

Model	Budget	Differential retained	Uniform revert
Meta-Llama-3-8B-Instruct	50%	2 / 6	4 / 6
Meta-Llama-3-8B-Instruct	70%	5 / 6	1 / 6
Llama-2-7B	50%	4 / 6	2 / 6
Llama-2-7B	70%	6 / 6	0 / 6
Qwen2.5-7B-Instruct	50%	3 / 6	2 / 6, 1 tie
Qwen2.5-7B-Instruct	70%	5 / 6	1 / 6
Mistral-7B-v0.1	50%	2 / 6	4 / 6
Mistral-7B-v0.1	70%	3 / 6	3 / 6
Llama-2-13B	70%	5 / 6	1 / 6

C.2 Finalized Retained Configurations

The tables below list the finalized retained configurations used in the sparse-only reporting. For readability, we report the target proportion distribution $[p_0, p_1, p_2]$, the corresponding sparsity levels $[\rho_0, \rho_1, \rho_2]$, and the final reporting decision for each task. Tasks marked as "uniform revert" use the matched-budget uniform baseline in the final reported result.

D Additional Joint Sparse-Quant Results

This appendix provides additional task-level results for the joint sparse-quant comparison summarized in Table 5. Since Meta-Llama-3-8B-Instruct is already reported in detail in Table 4, we focus here on the additional model families. For each model, we report results on the same six focal LongBench tasks used in the main text: NarrativeQA, Qasper, MultiFieldQA-EN, HotpotQA, TREC, and LCC.

Overall, the task-level breakdowns are consistent with the trend reported in the main text. The gains of JSQKV over the sequential M+K baseline are more visible under tighter compression budgets, whereas the milder 50/50 settings remain more mixed. In particular, improvements often concentrate on a subset of tasks rather than appearing uniformly across all six tasks.

Table 16. Finalized sparse-only retained configurations for Meta-Llama-3-8B-Instruct [7].

Budget	Task	Final decision	$[p_0, p_1, p_2]$	$[\rho_0, \rho_1, \rho_2]$	Notes
50%	narrativeqa	uniform revert	–	–	revert to matched-budget uniform
50%	qasper	uniform revert	–	–	revert to matched-budget uniform
50%	multifieldqa_en	uniform revert	–	–	revert to matched-budget uniform
50%	hotpotqa	uniform revert	–	–	revert to matched-budget uniform
50%	trec	differential	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
50%	lcc	differential	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
70%	narrativeqa	uniform revert	–	–	revert to matched-budget uniform
70%	qasper	differential	[0.0, 0.857143, 0.142857]	[0.0, 0.65, 1.0]	value_aware, mean, sink=4
70%	multifieldqa_en	differential	[0.02, 0.8, 0.18]	[0.0, 0.65, 1.0]	value_aware, max, sink=4
70%	hotpotqa	differential	[0.0, 0.75, 0.25]	[0.0, 0.6, 1.0]	default settings
70%	trec	differential	[0.0, 0.75, 0.25]	[0.0, 0.6, 1.0]	default settings
70%	lcc	differential	[0.0, 0.75, 0.25]	[0.0, 0.6, 1.0]	default settings

Table 17. Finalized sparse-only retained configurations for Llama-2-7B [16].

Budget	Task	Final decision	$[p_0, p_1, p_2]$	$[\rho_0, \rho_1, \rho_2]$	Notes
50%	narrativeqa	differential	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
50%	qasper	uniform revert	–	–	revert to matched-budget uniform
50%	multifieldqa_en	differential	[0.05, 0.75, 0.20]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
50%	hotpotqa	uniform revert	–	–	revert to matched-budget uniform
50%	trec	differential	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
50%	lcc	differential	[0.0, 1.0, 0.0]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	narrativeqa	differential	[0.1, 0.444444, 0.455556]	[0.0, 0.55, 1.0]	value_aware, mean, sink=8
70%	qasper	differential	[0.1, 0.444444, 0.455556]	[0.0, 0.55, 1.0]	value_aware, max, sink=2
70%	multifieldqa_en	differential	[0.15, 0.375, 0.475]	[0.0, 0.6, 1.0]	value_aware, max, sink=2
70%	hotpotqa	differential	[0.15, 0.6, 0.25]	[0.0, 0.75, 1.0]	value_aware, max, sink=2
70%	trec	differential	[0.0, 0.6, 0.4]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	lcc	differential	[0.0, 0.666667, 0.333333]	[0.0, 0.55, 1.0]	value_aware, max, sink=2

Table 18. Finalized sparse-only retained configurations for Qwen2.5-7B-Instruct [19].

Budget	Task	Final decision	$[p_0, p_1, p_2]$	$[\rho_0, \rho_1, \rho_2]$	Notes
50%	narrativeqa	uniform revert	–	–	revert to matched-budget uniform
50%	qasper	differential	[0.15, 0.7, 0.15]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
50%	multifieldqa_en	uniform revert	–	–	revert to matched-budget uniform
50%	hotpotqa	differential tie	[0.0, 1.0, 0.0]	[0.0, 0.5, 1.0]	final full result ties uniform
50%	trec	differential tie	[0.0, 1.0, 0.0]	[0.0, 0.5, 1.0]	final full result ties uniform
50%	lcc	differential	[0.05, 0.9, 0.05]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	narrativeqa	differential	[0.0, 0.75, 0.25]	[0.0, 0.6, 1.0]	value_aware, max, sink=2
70%	qasper	differential	[0.0, 0.75, 0.25]	[0.0, 0.6, 1.0]	value_aware, max, sink=2
70%	multifieldqa_en	differential	[0.1, 0.4, 0.5]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	hotpotqa	differential	[0.0, 0.6, 0.4]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	trec	differential	[0.0, 0.6, 0.4]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	lcc	uniform revert	–	–	revert to matched-budget uniform

Table 19. Finalized sparse-only retained configurations for Mistral-7B-v0.1 [9].

Budget	Task	Final decision	$[p_0, p_1, p_2]$	$[\rho_0, \rho_1, \rho_2]$	Notes
50%	narrativeqa	uniform revert	–	–	revert to matched-budget uniform
50%	qasper	differential	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
50%	multifieldqa_en	differential	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	value_aware, max, sink=2
50%	hotpotqa	uniform revert	–	–	revert to matched-budget uniform
50%	trec	differential tie	[0.0, 0.833333, 0.166667]	[0.0, 0.4, 1.0]	final full result ties uniform
50%	lcc	uniform revert	–	–	revert to matched-budget uniform
70%	narrativeqa	uniform revert	–	–	revert to matched-budget uniform
70%	qasper	differential	[0.025, 0.785714, 0.189286]	[0.0, 0.65, 1.0]	value_aware, mean, sink=2, prr=1.0
70%	multifieldqa_en	differential	[0.05, 0.625, 0.325]	[0.0, 0.6, 1.0]	value_aware, max, sink=2
70%	hotpotqa	uniform revert	–	–	revert to matched-budget uniform
70%	trec	differential	[0.0, 0.6, 0.4]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	lcc	uniform revert	–	–	revert to matched-budget uniform

Table 20. Finalized sparse-only retained configurations for Llama-2-13B. Only 70% results are reported at task level in the current draft because full-task 50% follow-ups remain incomplete.

Budget	Task	Final decision	$[p_0, p_1, p_2]$	$[\rho_0, \rho_1, \rho_2]$	Notes
70%	narrativeqa	differential	[0.05, 0.714286, 0.235714]	[0.0, 0.65, 1.0]	value_aware, max, sink=2
70%	qasper	differential	[0.0, 0.75, 0.25]	[0.0, 0.6, 1.0]	value_aware, max, sink=2
70%	multifieldqa_en	differential	[0.0, 0.857143, 0.142857]	[0.0, 0.65, 1.0]	value_aware, max, sink=2
70%	hotpotqa	uniform revert	–	–	revert to matched-budget uniform
70%	trec	differential	[0.0, 0.6, 0.4]	[0.0, 0.5, 1.0]	value_aware, max, sink=2
70%	lcc	differential	[0.15, 0.428571, 0.421429]	[0.0, 0.65, 1.0]	value_aware, max, sink=2

Table 21. Task-level joint sparse-quant results on Llama-2-7B [16].

Setting	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
M+K, 50/50 + 4-bit	15.91	**9.26**	21.91	7.45	**66.00**	67.49	31.34
JSQKV, 50/50 + 4-bit	**15.93**	8.70	**22.51**	**7.58**	**66.00**	**67.50**	**31.37**
M+K, 50/50 + 2-bit	**17.95**	**9.66**	**23.49**	**7.47**	65.50	65.00	**31.51**
JSQKV, 50/50 + 2-bit	13.53	8.58	20.13	7.13	**66.00**	**66.00**	30.23
M+K, 70/70 + 4-bit	**15.13**	**8.29**	**22.04**	**7.56**	64.50	65.91	30.57
JSQKV, 70/70 + 4-bit	14.70	**8.29**	21.89	7.33	**66.00**	**67.60**	**30.97**
M+K, 70/70 + 2-bit	**15.96**	7.68	**21.29**	7.05	63.50	64.01	29.92
JSQKV, 70/70 + 2-bit	15.07	**8.38**	18.97	**7.29**	**65.50**	**65.33**	**30.09**

Table 22. Task-level joint sparse-quant results on Mistral-7B-v0.1 [9].

Setting	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
M+K, 50/50 + 4-bit	13.12	8.47	**30.73**	**9.44**	**69.00**	**66.31**	32.84
JSQKV, 50/50 + 4-bit	**13.60**	**8.91**	30.69	9.37	**69.00**	65.84	**32.90**
M+K, 50/50 + 2-bit	**15.63**	7.73	27.73	9.60	66.50	63.04	31.70
JSQKV, 50/50 + 2-bit	14.31	**8.42**	**27.97**	**9.85**	**68.00**	**65.65**	**32.37**
M+K, 70/70 + 4-bit	12.64	8.18	29.96	9.37	**69.00**	65.44	32.43
JSQKV, 70/70 + 4-bit	**12.84**	**8.36**	**30.79**	**9.61**	68.50	**66.14**	**32.71**
M+K, 70/70 + 2-bit	12.17	7.11	24.22	**9.52**	64.50	63.35	30.14
JSQKV, 70/70 + 2-bit	**14.67**	**7.99**	**28.17**	9.50	**69.00**	**64.79**	**32.35**

Table 23. Task-level joint sparse-quant results on Qwen2.5-7B-Instruct [19].

Setting	NarrativeQA	Qasper	MultiFieldQA_EN	HotpotQA	TREC	LCC	Avg.
M+K, 50/50 + 4-bit	15.37	17.82	26.34	27.20	64.50	35.82	31.17
JSQKV, 50/50 + 4-bit	**17.64**	**29.76**	**34.02**	**31.52**	**65.00**	**44.53**	**37.08**
M+K, 50/50 + 2-bit	**13.13**	13.32	20.62	21.40	**59.25**	**28.90**	**26.10**
JSQKV, 50/50 + 2-bit	11.84	**16.57**	**23.41**	**23.62**	46.00	19.44	23.48
M+K, 70/70 + 4-bit	17.11	19.54	30.24	29.83	62.00	39.28	33.00
JSQKV, 70/70 + 4-bit	**19.11**	**28.61**	**40.84**	**33.28**	**65.50**	**50.66**	**39.67**
M+K, 70/70 + 2-bit	11.98	14.42	20.44	23.06	57.25	25.90	25.51
JSQKV, 70/70 + 2-bit	**17.88**	**26.45**	**30.99**	**30.76**	**59.00**	**46.13**	**35.20**